

План: маски, большие канвасы, несколько схем на сцену, запуск с D: (v2.6)

Составлен 2026-09-30 по запросу пользователя (8 пунктов, позже добавлен 9-й). Роль автора плана — только план; реализуют другие агенты по промптам из раздела «Промпты».

Черновики классов (НЕ компилируются, это `.java.txt`) — `drafts/: MaskGeometry.java.txt`, `CanvasFit.java.txt` (трек A), `StreamingPngWriter.java.txt` (трек B). Можно брать за основу, проверив заново.

1. Исходные запросы пользователя (дословно по смыслу)

1. Кнопка «задать размер шрифта для всей схемы» — для всех блоков с дефолтным размером; блоки с кастомным размером не трогать.
2. Убрать ограничение 16k на сторону при генерации масок и канвасов.
3. Убрать колонку/настройку «Растр» (не используется); вместо неё — видимость плашки с именем экрана; отдельная галочка для разрешения на той же плашке.
4. Параметр маски «сетка»: экран из кабинетов-сеток, маска в 2 раза выше.
5. Предупреждение, если канвас меньше размещённых в нём масок (обрезает их).
6. Своя пара цветов для маски; механизм палитры из общих схем, но память цветов масок — отдельная.
7. Программа не открывается, если папку положить на диск D: — почему, как исправить.
8. Несколько блок-схем на сцену. В режиме схемы верхние плашки «Проект» и «Сцена» как были, плашка «Экран» меняется на «Схема» со списком схем; окно редактирования названия схемы; в сетевом менеджере при импорте из общей схемы — выбор схемы сигнала.
9. Переделать сохранение своего блока в библиотеку: сейчас можно сохранить ПУСТОЙ пресет. Кнопку «Сохранить как пресет» справа убрать, перенести в меню ПКМ по блоку; сохранять в т.ч. все карты и разлёмы, которые пользователь добавил блоку на схеме.

2. Решения пользователя

#	Вопрос	Решение
D1	Спецификация при нескольких схемах	Каждая схема считается ОТДЕЛЬНО: одни и те же позиции пишутся в разных колонках, подписанных названием схемы. Какие блоки «общие» — узнать нельзя, перестраховываемся.
D2	Автозаполнение экранами/ контроллерами	Только в ПЕРВОЙ схеме режима.
D3	«Сетка»	Множитель высоты маски, по умолчанию 2.
D4	Веб-редактор масок на сервере	Повторить изменения масок (п. 3, 4, 6). Деплой — только с явного разрешения.
D5	Старые клиенты открывают новый файл	Схемы одного режима хранятся в файле со сдвигом по X (ширина предыдущих + зазор); новый клиент сдвиг вычитает. Старый клиент видит схемы рядом, а не стопкой.
D6	Спецификация и сцены	Для КАЖДОЙ сцены проекта — отдельная спецификация (свой .xlsx, все листы только по этой сцене); колонки по схемам — внутри спецификации сцены. Общепроектной спецификации больше нет.

D7	Лимит 16k	Реализуем три меры: потоковая запись полосами, увеличенный heap, уменьшенное превью. Сторона маски/канваса ≥ 16384 px — предупреждение; жёсткое ограничение — 30000 px. Разрешение превью — пресеты как в After Effects (Полное, 1/2, 1/3, 1/4, 1/8, Авто).
----	-----------	--

Колонки «Итого» по схемам нет — осознанно (суть D1): итог по нескольким схемам вводил бы в заблуждение, если одни и те же блоки нарисованы на нескольких схемах.

3. Установленные факты (проверено по коду 2026-09-30)

- Лимит 16k — только спиннеры `VisualizationStagePanel:148-149 (SpinnerNumberModel(..., 16384, ...))`. Реальные ограничения глубже: маски рисуются целиком в `BufferedImage` (`PixelGridRenderer.renderMask/renderCanvasMask`, 4 Б/px, маска экрана сначала отдельной картинкой, потом `drawImage` в канвас); `CanvasEditorPanel.maskCache` держит маски в ПОЛНОМ разрешении; диалог предпросмотра (`VisualizationStagePanel.showMaskPreviewDialog`) держит все изображения сразу. `ResumePresetExporter:65` пишет `max="16384"`; After Effects ограничивает композицию 30000×30000.
- Растр: `CanvasPlacement.showRaster` (по умолчанию true), рисуется в `PixelGridRenderer:137`, в т.ч. в отдельных масках экранов (`defaultForScreen`).
- Цвет маски: `Screen.background (MaskColorPreset)`, общий на экран во всех канвасах; читается в `PixelGridRenderer`, `Structure3DPanel:895-911`, комбобокс «Фон» в таблице гридов, комбобокс дефолтов сцены `SetupStagePanel:2630`. Миграция legacy per-placement цвета — `AppModel.seedScreenMaskColorsFromLegacyPlacements`.
- Палитра линий: `UIKit.showColorChooser(owner, title, initial, settings) + RecentColorsChooserPanel` (static список + `UserProfile.recentLineColors`, `SettingsManager.rememberRecentLineColor`). Тесты: `RecentColorsChooserPanel*Test`, `settings/RecentLineColorPersistenceTest`.
- `model.MaskColorPreset`, `Screen`, `Scene`, `SchemaNode/Edge` — КЛИЕНТСКИЕ классы (не `ledscheme-model`). JS-массив `MASK_PRESETS` в `ledscheme-server/src/main/resources/static/app.js` — ручная копия (золотое правило 4), там же свой редактор масок с галочкой растра.
- Схема: узлы/связи — плоские списки `Scene.schemaNodes/schemaEdges`, режим — поле `mode`. Доступ для UI — `AppModel.schemaNodesForCurrentScene(mode) / schemaEdgesForCurrentScene(mode)` (используют `SchemaPanel`, `SchemaCanvasPanel`, `NetworkManagerPanel`, `schemalayout/SchemaUsage`). Прямые `currentScene.getSchemaNodes()/getSchemaEdges()` — ~25 мест в `AppModel` (`add/paste/delete/clearSchema/orphans/autoPopulate/legends/networkGraphFromScene`), плюс `SchemaLoadCalc`, `NetworkCanvasPanel:1017`, `NetworkDeviceLabels:48`, `OutputStagePanel:772,1029,1060` (спецификация).
- Админка `ledscheme-admin/.../project/ProjectSummary.java` читает `scene.schemaNodes` из JSON напрямую — поэтому списки остаются плоскими.
- Все места чтения JSON в клиенте выключают `FAIL_ON_UNKNOWN_PROPERTIES` — старый клиент откроет файл с новыми полями, но при пересохранении их потеряет.
- `ContextBar` («Проект → Сцена → (Экран)») создаётся в `PowerStagePanel:212`, `SignalStagePanel:400` с `includeScreen=true`; вид переключается `schemaViewBtn / chainViewBtn` (+ `networkViewBtn` в «Сигнале»).
- Спецификация сейчас — ОДИН файл на проект `<Проект>_спецификация.xlsx` (`OutputStagePanel.buildEquipmentSpecWorkbook(project)`, ~строка 707; вызовы — пакет документации ~526 и кнопка «только спецификация» `exportSpecOnly` ~676); все листы («Инфо», «Кабинеты», «Оборудование», «Конструктив», «Фермы», «Общий список», «Коммутация — сводная/сплайсовка») суммируют ВСЕ сцены циклом `for (Scene scene : project.getScenes())`. Пакет документации уже раскладывает файлы по папкам сцен (`<папка>/<Сцена>/Сила|Сигнал|Маски`, ~383).
- «Сохранить как пресет» (п. 9): кнопка `saveAsPresetBtn` в форме «Добавить узел» `SchemaPanel` (~строки 56, 188-190, включение в `updateAddFormEnablement` ~301), метод `SchemaPanel.saveAsPreset` (~437) вызывает `model.addEquipmentPreset(mode, type, name, "", null)` — берёт только подпись из поля ввода,

- карт нет вообще, отсюда пустые пресеты. Меню ПКМ блока — `SchemaCanvasPanel.showNodeMenu`.
- Обратное направление уже есть: `AppModel.addSchemaNodeFromPreset` копирует карты (`duplicateCardWithFreshIds`) и разъёмы питания (`duplicatePortWithFreshId`) пресета в узел. `EquipmentPreset` (в `ledscheme-model`) хранит карты и разъёмы питания, раскладку гнёзд (`PortPlacement`) — нет.
- «Перенести из схемы...»: `NetworkManagerPanel.openSchemaImportDialog` → `NetworkCanvasPanel.previewSchemaImport/applySchemaImport` → `AppModel.networkGraphFromScene(scene)`; диалог `SchemaImportDialog`.
 - Диск D: — в сборке нет привязки к C: (лаунчер `jpackage` использует `$APPDIR`, данные в `~/led-scheme` через `user.home`). Ошибки старта не видны: `App.main` делает `printStackTrace()` под `javaw`. Перезапуск/автообновление (`UpdateManager:248, 277`) зовут `javaw -jar` — в сборке `jpackage` это системная Java (если есть), а не встроенный рантайм.
 - В рабочем дереве **master** — большая незакоммиченная работа **v2.6**, и параллельно с планированием в том же дереве работала другая сессия (`ContextBar`, `EdgeBundles`, `SchemaCanvasPanel`). Работать агентам в одном дереве нельзя.

4. Организация работ

4.1 Предусловие (делает пользователь)

Wave 0: закоммитить текущую работу v2.6 в master (включая работу параллельной сессии) — иначе `worktree` агентов её не увидят, а правки в общем дереве перемешаются. Без этого шага волны не запускать.

4.2 Треки и волны

Трек	Пункты	Волна	Зависит от	Модель
A — маски	3, 4, 5, 6	1	—	Sonnet
C1 — модель схем	8 (модель, миграция, <code>AppModel</code> , сдвиг D5)	1	—	Opus
D — запуск/лог	7	1	—	Sonnet
F — пресет из блока	9	1	—	Sonnet
B — большие канвасы	2	2	A влит	Opus
C2 — UI схем	8 (<code>ContextBar</code> , диалог названия, стадии, экспорт)	2	C1 влит	Sonnet
C3 — спецификация и сетевой менеджер	8 (D1, D6, импорт)	2	C1 влит	Sonnet
C4 — шрифт схемы	1	3	C2 влит	Sonnet
E — веб-редактор масок	D4	3	A влит	Sonnet
I — интеграция	—	4	все	Opus

Пересечения файлов, которые учтены порядком волн: `PixelGridRenderer` / `VisualizationStagePanel` / `CanvasEditorPanel` (A→B), `SchemaCanvasPanel` / `SchemaPanel` (C1→C2→C4; F в волне 1 правит только кнопку пресета и один пункт меню блока — мелкий мерж), `OutputStagePanel` (B — экспорт масок, C2 — экспорт изображений схем, C3 — спецификация: разные методы, мерж без конфликтов по смыслу), `AppModel` (A — секция масок, C1 — секция схем), `UserProfile` / `SettingsManager` (только A).

4.3 Правила для всех агентов (вставлены в каждый промпт как «Общие правила»)

- Работа в СВОЁМ git worktree от актуального master, ветка `claude/<трек>`.
- Коммиты в свою ветку разрешены пользователем при запуске агента; push, merge в master, теги, деплой — НЕТ.
- До начала: прочитать `CLAUDE.md`, этот план (§2, §3, свой трек), вызвать скилл `run-tests`; при необходимости — `test-build`.
- `testbuild/led-scheme-test.jar` — общий для всех worktree: агенты волн его НЕ перезаписывают (гонка между параллельными агентами); сборку проверяют `mvn -DskipTests package` в своём worktree. Тестовый билд пересобирает трек I после мержа. (Осознанное исключение из правила `CLAUDE.md` на время волн.)
- Полный `mvn test` в своём worktree — 0 failures перед завершением.
- Стил: javadoc объясняет ПОЧЕМУ (запрос/баг-репорт с датой 2026-09-30), как в окружающем коде; русские строки UI; не вычищать старые комментарии.
- Старые проекты должны открываться: новые поля — с дефолтами, сохраняющими прежнее поведение, кроме явно заказанных изменений.
- Не трогать файлы чужих треков сверх необходимого; если пришлось — перечислить в отчёте.
- Отчёт в конце: что сделано, список файлов, новые тесты, результат `mvn test`, открытые вопросы, отклонения от плана. Строки для `CHANGELOG.md` / `TESTS.md` дать текстом в отчёте (сами файлы правит трек I, чтобы не было конфликтов).

5. Промпты

Общие правила (вставляется в начало каждого промпта)

Проект: C:\Development\VJsTB_reborn (десктоп-клиент AVE_ToolBox, Java 21, Swing, Maven). Прочитай `CLAUDE.md` и `docs/masks-and-schema-sheets/PLAN.md` (§2 решения пользователя, §3 факты, §4.3 правила, свой трек в §5).

- Работай в своём git worktree от master, ветка `claude/<трек>`. Коммиты в эту ветку разрешаю; push/merge/теги/деплой — нет.
- Вызови скилл `run-tests` перед запуском тестов. Не перезаписывай `testbuild/led-scheme-test.jar` — сборку проверяй `mvn -DskipTests package`.
- Перед завершением: полный `mvn test`, 0 failures.
- Javadoc в стиле проекта (объясняет ПОЧЕМУ, со ссылкой на запрос 2026-09-30).
- Старые проекты должны открываться без изменений поведения, кроме заказанного.
- `CHANGELOG.md` / `TESTS.md` не правь — дай готовые строки в отчёте.
- Отчёт: сделано / файлы / тесты / результат `mvn test` / вопросы / отклонения.

Трек А — маски (пункты 3, 4, 5, 6)

<Общие правила>

Задача — четыре доработки этапа «Генерация масок». Основные файлы: `model/CanvasPlacement`, `model/Screen`, `model/MaskColorPreset`, `ui/PixelGridRenderer`, `ui/Stage/VisualizationStagePanel` (таблица гридов `PlacementsTableModel`), `ui/CanvasEditorPanel`, `ui/ResolumePresetExporter`, `ui/AfterEffectsJsxWriter`, `ui/Structure3DPanel`, `ui/Stage/SetupStagePanel` (комбобокс дефолтов ~2630), `ui/RecentColorsChooserPanel`, `ui/UiKit`, `settings/UserProfile`, `settings/SettingsManager`, `service/AppModel` (секция масок ~4160-4230). Черновики: `docs/masks-and-schema-sheets/drafts/MaskGeometry.java.txt` и `CanvasFit.java.txt`.

- 1) Убрать «Растр». Колонку «Растр» удалить, растр не рисовать нигде (включая отдельные маски экранов). `CanvasPlacement.showRaster` оставить `@Deprecated` только для чтения старого JSON. Вместо колонки — две: «Плашка» (`showNameLabel`) и

«Разрешение» (showResolution), обе по умолчанию true. Плашка рисуется, если включена хотя бы одна; при одной строке высота плашки под одну строку. Замени 9-аргументный AppModel.updatePlacementMaskConfig на updatePlacementMask(CanvasPlacement, Consumer<CanvasPlacement>) – одна запись отмены.

- 2) «Сетка» (решение D3: множитель, по умолчанию 2). На Screen: boolean maskMesh, int maskHeightMultiplier = 2 (min 1), effectiveMaskHeightMultiplier() (@JsonIgnore). Свойство ЭКРАНА (одинаково во всех канвасах, как цвет). В таблице гридов – колонки «Экран-сетка» (чекбокс) и «Высота x» (целое, редактируемо только при включённой сетке). Не путать с существующей колонкой «Сетка» (showGrid – рамки кабинетов), её не переименовывать. Введи service/MaskGeometry (см. черновик): размер маски и ячейки с учётом множителя, cabinetX/cabinetY с мм-смещениями. ВСЕ потребители берут размеры маски только через неё: renderMask, renderCanvasMask, renderCanvasGapMask, renderCanvasOverlay, CanvasEditorPanel (отрисовка, hit-test, Shift-прилипание, HUD-подпись), ResolumePresetExporter (размер слайса), AfterEffectsJsxWriter (размер прекомпа). Подпись разрешения на плашке – размер маски с пометкой «(сетка xN)». Расчёты сигнала/питания не трогать.
- 3) Предупреждение «канвас обрезает маски». service/CanvasFit (см. черновик): размещение вылезает, если x+ширина_маски > W или y+высота_маски > H. Показать: (а) строку-предупреждение на этапе под выбором канваса (цвет Palette предупреждения, список экранов, обновляется по model listener); (б) в CanvasEditorPanel – красная обводка/штриховка вылезающей части; (в) подтверждение перед «Экспорт масок», «Экспорт текущего канваса», Resolume и AE, если есть переполнение в экспортируемых канвасах; (г) предупреждение при «Применить размер», если НОВЫЙ размер обрежет маски.
- 4) Своя пара цветов. MaskColorPreset.CUSTOM («Свои цвета...», запасные цвета = NORMAL); Screen.maskColorA/maskColorB (Integer RGB); Screen.maskColor(parity) (@JsonIgnore) – единственный способ получить цвет клетки (PixelGridRenderer, Structure3DPanel). GridRenderOptions несёт пару Color вместо пресета. AppModel.setMaskCustomColors(screen, a, b) – ставит CUSTOM. В таблице «Фон»: выбор «Свои цвета...» открывает диалог с двумя образцами (Цвет 1, Цвет 2) и превью шахматки; образец открывает UIKit.showColorChooser. Рендерер ячейки показывает пару цветов. В комбобоксе дефолтов сцены (SetupStagePanel) CUSTOM не показывать. ОТДЕЛЬНАЯ память цветов масок: RecentColorsChooserPanel получает «канал» (enum LINES/MASKS) со своим static-списком; UserProfile.recentMaskColors + SettingsManager.rememberRecentMaskColor; перегрузка UIKit.showColorChooser(..., channel); существующие вызовы остаются на LINES без изменений. JS-копию MASK_PRESETS на сервере НЕ трогать (трек E).

Тесты (JUnit 5, по образцу ui/PixelGridRendererCanvasMaskSceneTest и AppModelTest): маска без раstra (пиксель внутри кабинета без линий 16px); плашка выключена / только разрешение; маска экрана-сетки = Wx(H·N), ячейки и маска пустот xN; CanvasFit – вылезает/не вылезает/учёт множителя; слайс Resolume и размер в jsx с множителем; CUSTOM-цвет в пикселе маски; старый JSON без новых полей → showNameLabel/showResolution true, maskMesh false, multiplier 2; отдельная персистентность recentMaskColors и recentLineColors; setMaskCustomColors + undo.

Вне задачи: снятие лимита 16k и потоковая запись (трек B), веб-страница (трек E). В отчёте дай текст для GuideDialog (раздел про маски) – сам GuideDialog не правь.

Трек B — канвасы и маски больше 16k (пункт 2, решение D7)

<Общие правила>

Предусловие: трек A влит в master (MaskGeometry, GridRenderOptions без раstra, CanvasFit).

Задача (решение пользователя D7): снять лимит 16k, не раздувая память.

Сторона маски/канваса ≥ 16384 px – ПРЕДУПРЕЖДЕНИЕ, жёсткий предел – 30000 px (совпадает с пределом композиции After Effects). Превью – с пресетами разрешения как в After Effects.

Файлы: ui/PixelGridRenderer, ui/stage/VisualizationStagePanel, ui/CanvasEditorPanel, ui/stage/OutputStagePanel (экспорт масок в пакете документации ~строки 420 и 475), ui/ResolumePresetExporter, ui/AfterEffectsJsxWriter, service/AppModel (addCanvas/updateCanvas ~3945), settings/UserProfile + SettingsManager (настройка превью), новый ui/StreamingPngWriter (черновик: docs/masks-and-schema-sheets/drafts/StreamingPngWriter.java.txt), новый service/MaskLimits, .claude/skills/release-client/SKILL.md и .github/workflows/release-linux.yml, release-macos.yml (флаги jpackage).

- 1) Пределы – одна точка: service/MaskLimits { WARN_SIDE_PX = 16384; MAX_SIDE_PX = 30000; }, с javadoc «почему» (16384 – типовой предел текстуры GPU/медиасерверов и Resolume; 30000 – предел композиции After Effects и решение пользователя D7) и методами-проверками для размера w×h.
 - Канвас: спиннеры ширины/высоты – максимум MAX_SIDE_PX (вместо 16384); AppModel.addCanvas/updateCanvas отклоняют > MAX_SIDE_PX IllegalArgumentException с русским текстом (UI показывает его, как сейчас). Рядом со спиннерами – строка-предупреждение, как только введённая сторона ≥ WARN_SIDE_PX (обновляется при вводе); при «+ Новый канвас»/«Применить размер» с такой стороной – подтверждение.
 - Маски экранов: размер экрана задаётся на «Сетапе», запретить его здесь нельзя. Маска экрана со стороной ≥ WARN_SIDE_PX – предупреждение (значок/подсветка строки в таблице гридов + подсказка); > MAX_SIDE_PX – такая маска НЕ экспортируется: перед экспортом список таких экранов и понятное сообщение, остальные файлы пишутся. Учитывать множитель «сетки» (размеры только через MaskGeometry).
 - Экспорт (маски, текущий канвас, Resolume, AE, пакет документации): если в наборе есть что-то ≥ WARN_SIDE_PX – одно общее подтверждение со списком («имя» – W×H px). ResolumePresetExporter: убрать жёстко зашитый max="16384", писать max(16384, размер) – сверить формат с ResolumePresetExporterTest.
 - 2) Рендер через paint. PixelGridRenderer переделать на рисование в Graphics2D в координатах изображения: paintMask(g, screen, type, ws, opts) – маска экрана в локальных координатах; paintCanvasMask/GapMask/Overlay – канвас, маски экранов рисуются прямо в него (translate + clip по прямоугольнику экрана, чёрная подложка под скрытые кабинеты как раньше), без промежуточных BufferedImage. Кабинеты/размещения вне текущего clip пропускать (скорость). renderMask/renderCanvasMask/renderCanvasGapMask/renderCanvasOverlay (возвращают BufferedImage) оставить как обёртки для тестов и мелких размеров.
 - 3) Абстракция задания: MaskImage (width, height, alpha, paint(g)) с методами render(scale) – изображение в масштабе (через g.scale), thumbnail(maxW, maxH) и writePng(File) – полосами по ~512-1024 строк через StreamingPngWriter (память ∝ ширине). Поточный PNG должен совпадать попиксельно с renderX()+ImageIO на небольшом канвасе – главный тест.
 - 4) Превью с пресетами (как «Resolution» в After Effects). Комбобокс «Разрешение превью»: Полное, 1/2, 1/3, 1/4, 1/8, Авто – на панели этапа «Генерация масок» рядом с холстом; значение сохраняется в профиле (UserProfile.maskPreviewResolution, по умолчанию «Авто»).
 - Авто – маска рисуется ровно в размере отображения на экране.
 - Дробь – маска рисуется в доле нативного разрешения и масштабируется при показе (видны реальные потери детализации, как в AE).
 - Бюджет памяти: если превью одного размещения при выбранной дроби больше ~64 Мпикс, автоматически взять следующую меньшую дробь и показать в углу холста пометку «превью понижено до 1/N». Полное разрешение 30000×30000 в превью никогда не держим целиком.
 - CanvasEditorPanel.maskCache: ключ – id размещения + фактический масштаб рендера; сброс при смене пресета/масштаба окна (как сейчас при изменении модели).
 - Диалог предпросмотра экспорта хранит задания (MaskImage) и маленькие миниатюры, а не полные картинки; двойной клик/кнопка «Просмотр» открывает окно с прокруткой в выбранном пресете (Авто = вписать в окно); запись на диск – только по «Сохранить», потоково.
 - Все экспорты (маски, канвасы, AE-маски, пустоты, оверлей, пакет документации) пишут через writePng. Тяжёлый экспорт – в SwingWorker с ExportProgressDialog (по образцу OutputStagePanel), UI не блокировать.
- 5) Память JVM: в SKILL release-client (шаг 6, команда jpackage) и в обоих workflow (.deb/.dmg) добавить --java-options "-XX:MaxRAMPercentage=75" с

одной строкой «почему». Сам релиз не собирать, workflow не запускать.

Тесты: MaskLimits (граничные 16383/16384/30000/30001); AppModel.addCanvas/updateCanvas отклоняют 30001; StreamingPngWriter – валидный PNG, читается ImageIO, RGB и RGBA, несколько IDAT; потоковая запись канваса == renderCanvasMask по픽сельно (несколько полос, экран пересекает границу полосы, текст плашки на границе); то же для маски пустот (RGBA) и оверлея; render(1/4) даёт ровно ¼ размера; автопонижение пресета при превышении бюджета (чистая функция выбора дроби); список «≥16k»/«>30k» для набора экспорта с учётом множителя сетки; Resolume XML с канвасом 20000 px.

Трек C1 — несколько схем: модель, миграция, AppModel (пункт 8, часть 1)

<Общие правила>

Задача: модельный слой «несколько блок-схем на сцену». UI – не твой (трек C2), спецификация и сетевой менеджер – трек C3, шрифт схемы – трек C4. Файлы: model/Scene, model/SchemaNode, model/SchemaEdge, новый model/SchemaSheet, service/AppModel (секция схем ~1465-2300, autoPopulateSchema ~5485, легенды ~6130-6330, networkGraphFromScene ~6390), service/SchemaLoadCalc, service/schemalayout/SchemaUsage, store/WorkspaceStore (при необходимости).

- 1) Модель. SchemaSheet {id, name, mode (SchemaMode), orderIndex, Integer defaultFontSize, Integer defaultEdgeFontSize (оба null; используются треком C4), storageOffsetX (служебное, см. п.4)}. Scene.schemaSheets (List, не null). SchemaNode.sheetId, SchemaEdge.sheetId; copy() переносит. Узлы/связи остаются ПЛОСКИМИ списками в Scene (админка ProjectSummary читает scene.schemaNodes).
- 2) Миграция/инвариант: у каждой сцены на каждый режим ≥ 1 схема («Схема питания», «Схема сигнала»); узлы/связи без sheetId или с sheetId несуществующей схемы – в первую схему своего режима. Выполнять при загрузке (по образцу seedScreenMaskColorsFromLegacyPlacements, без undo) и лениво при создании сцены.
- 3) AppModel. Текущая схема на режим (transient; при смене сцены – первая схема). API: schemaSheets(mode) (по orderIndex), currentSchemaSheet(mode), selectSchemaSheet(sheet), addSchemaSheet(mode, name), renameSchemaSheet(sheet, name) (не пустое, уникальное в режиме сцены – иначе IllegalArgumentException с русским текстом), deleteSchemaSheet(sheet) (последнюю схему режима – нельзя; удаляет её узлы/связи и ссылки сетевого менеджера на её узлы – посмотри, как это делает deleteSchemaNodes), duplicateSchemaSheet(sheet) (новые id узлов, связей, гнёзд не менять – перепривязать from/to связей), moveSchemaSheet (порядок). Все – с pushUndo и changed(). schemaNodesForCurrentScene/schemaEdgesForCurrentScene(mode) фильтруют по ТЕКУЩЕЙ схеме. Все места, добавляющие узлы/связи (addSchemaNode*, pasteSchemaNodes, addSchemaEdge*, addSignalPortLegendNode, addLineLegendNode, addSchemaNodeFromPreset/NetworkDevice...), ставят sheetId текущей схемы. clearSchema(mode) – только текущая схема. autoPopulateSchema(mode) – только в ПЕРВУЮ схему режима (решение D2), независимо от текущей. Синхронизация гнёзд экранов с цепочками и удаление осиротевших узлов (~4227) – по всем схемам сцены, как сейчас. Легенды линий (lineLegendRoles/PowerNominals) и SchemaLoadCalc (нагрузки распределения) – в пределах СХЕМЫ узла, а не всей сцены. networkGraphFromScene(scene) → networkGraphFromSheet(scene, sheetId) (старую сигнатуру оставить, если есть внешние вызовы, делегируя в первую схему сигнала – вызовы переведёт C3).
- 4) Совместимость со старыми клиентами (решение D5). В JSON схемы одного режима раскладываются по X без перекрытия: offset(первая)=0, offset(k) = offset(k-1) + ширина(k-1) + 400, где ширина – правый край самого дальнего узла (x+width) или точки излома связи этой схемы (минимум 0). Сдвигаются SchemaNode.x и EdgeWaypoint.x; labelDx/Dy и гнёзда – нет. В памяти координаты каждой схемы локальные. Сделать это на уровне Jackson в Scene, чтобы покрыть ВСЕ пути сериализации (WorkspaceStore, экспорт/импорт проекта, облако, LocalArchiveStore, ClientBackup, история версий): @JsonProperty-геттеры schemaNodes/schemaEdges/schemaSheets отдадут сдвинутые КОПИИ и сохранённые offsets (один детерминированный расчёт на всех трёх), сеттеры принимают «сырые» значения, а вычитание offset выполняется один раз после десериализации (ленивый флаг при первом обращении или явный post-load –

выбери надёжный и опиши в javadoc). Файл без schemaSheets (старый клиент пересохранил) – обычная миграция п.2, ничего не вычитать. Проверь, что Jackson-снимки внутри приложения (если undo/копирование сцен используют convertValue/копирование через JSON) остаются тождественными.

Тесты: миграция старого JSON (узлы без sheetId → первая схема режима; по схеме на режим); CRUD + ограничения (последняя, дубли имён) + undo; фильтрация schema*ForCurrentScene по текущей схеме; добавление узла/связи/вставка идут в текущую схему; clearSchema чистит только текущую; autoPopulate только в первую; дубликат схемы с перепривязкой связей; SchemaLoadCalc и легенда линий не видят узлы соседней схемы; сохранить→загрузить – координаты не изменились; в сохранённом JSON схемы одного режима не пересекаются по X и точки излома сдвинуты; JSON без schemaSheets грузится без вычитания.

Трек C2 — UI нескольких схем (пункт 8, часть 2)

<Общие правила>

Предусловие: C1 влит (API схем в AppModel – прочитай его javadoc).

Задача: интерфейс выбора и управления схемами. Файлы: ui/ContextBar, ui/stage/PowerStagePanel, ui/stage/SignalStagePanel, ui/SchemaPanel (только если нужно реагировать на смену схемы), ui/stage/CurrentSchemeExporter, ui/stage/OutputStagePanel (экспорт изображений общих схем в пакете документации ~448-470), ui/GuideDialog (текст раздела про общую схему), новый диалог названия схемы.

- 1) ContextBar: режим «схема». Когда на этапе показан вид общей схемы, подпись и комбобокс «Экран:» заменяются на «Схема:» со списком схем текущего режима (по порядку) + кнопки «+» (новая, сразу диалог названия), «↵» (переименовать), «⌘» (дублировать), «⌫» (удалить, с подтверждением; для последней схемы – неактивна с подсказкой). «Проект» и «Сцена» не меняются. API вида setSchemaMode(SchemaMode или null). PowerStagePanel/SignalStagePanel переключают его вместе с кнопками вида (в «Сетевом менеджере» – обычный режим с экраном). Выбор в комбобоксе → model.selectSchemaSheet. Учти, что ContextBar недавно получил расширение попапа (widenPopupToFitContents) – применить и к комбобоксу схем.
- 2) Окно редактирования названия схемы: небольшой модальный JDialog (поле, OK/Отмена, Enter/Esc, валидация «не пусто и уникально в режиме» с текстом ошибки под полем, выделенный текст при открытии). Использовать для «+» и «↵».
- 3) Экспорт: «Экспорт схемы...» – текущая схема, имя файла по умолчанию «<Сцена> <Схема>». Пакет документации – каждая схема режима отдельным файлом с её именем (сейчас одна «Общая схема» на режим).
- 4) GuideDialog: абзац про несколько схем, «Схема:» в верхней строке, первая схема заполняется автоматически.

Тесты: логика ContextBar в режиме схемы (headless, по образцу существующих UI-тестов: какие комбобоксы видны, список схем, выбор меняет текущую схему); валидация диалога названия (вынеси проверку в тестируемый статический метод); имена файлов экспорта для нескольких схем.

Трек C3 — спецификация по схемам и импорт в сетевой менеджер (пункт 8, часть 3)

<Общие правила>

Предусловие: C1 влит.

- 1) Спецификация по сценам (решение D6). buildEquipmentSpecWorkbook(project) → buildEquipmentSpecWorkbook(project, scene): ВСЕ листы («Инфо», «Кабинеты», «Оборудование», «Конструктив», «Фермы», «Общий список», «Коммутация – сводная/сплайсовка») считаются только по этой сцене; в «Инфо» – строка «Сцена». Файл на сцену: `<папка>/<Сцена>/<Проект>\_<Сцена>\_спецификация.xlsx` (та же папка сцены, что уже создаёт пакет документации, ~строка 383) – и в пакете документации, и в кнопке «только спецификация» (exportSpecOnly). Общепроектный файл `<Проект>\_спецификация.xlsx` больше не пишется. Итоговые

диалоги перечисляют все файлы спецификаций. Счётчик шагов прогресса (total, ~строка 370) – +1 на сцену вместо +1 на проект. Сцена без экранов и схем – спецификацию всё равно писать (пустые листы), чтобы набор файлов был предсказуем; если решишь иначе – обоснуй в отчёте.

Внутри спецификации сцены (решение D1) каждая схема считается отдельно: в «Оборудование», «Коммутация – сводная», «Коммутация – сплайсовка» и строках «Коммутация» листа «Общий список» вместо одной колонки «Кол-во» – колонка на каждую схему сцены, подписанная её названием (порядок: схемы питания, затем сигнала, по `orderIndex`); одинаковые позиции разных схем – в одной строке, разные числа в разных колонках; позиции нет в схеме – пустая ячейка. Если имя схемы питания совпало с именем схемы сигнала – к заголовку добавить « (питание)»/« (сигнал)». Комплектация кусками кабеля (`CableSpecCalc`) – по каждой схеме отдельно. Колонку «Итого» НЕ добавлять. Вынеси агрегацию в тестируемый сервисный класс (без POI).

- 2) Сетевой менеджер: «Перенести из схемы...» – в `SchemaImportDialog` выбор схемы сигнала (комбобокс сверху, пересчёт предпросмотра при смене); если схема сигнала одна – выбор не показывать. `NetworkCanvasPanel.previewSchemaImport` принимает `sheetId`; вызывает `networkGraphFromSheet`. Проверь `NetworkCanvasPanel.networkDeviceTypeIdOfNode` и `NetworkDeviceLabels` – они ищут узел по `id` по всей сцене, это правильно, но убедись.
- 3) `GuideDialog`: строка про выбор схемы в «Перенести из схемы...» (абзац уже есть, ~строка 95).

Тесты: спецификация сцены не видит экраны/узлы/связи/конструктив ДРУГОЙ сцены; имена и расположение файлов по сценам; агрегация – две схемы с общим и различным оборудованием (две колонки, общая строка), совпадение имени схемы питания и сигнала (суффиксы), кабели кусками по схемам отдельно; импорт из выбранной схемы не видит устройства другой схемы.

Трек C4 — размер шрифта всей схемы (пункт 1)

<Общие правила>

Предусловие: C1 и C2 влиты. Поля `SchemaSheet.defaultFontSize/ defaultEdgeFontSize` уже есть (C1).

Задача: кнопка «Шрифт схемы...» в тулбаре `SchemaPanel` – задаёт размер по умолчанию для ТЕКУЩЕЙ схемы: блоки и подписи линий (два поля в одном диалоге, 0/пусто – стандартный; переиспользуй `promptFontSize` из `SchemaCanvasPanel` ~2841 или вынеси общий диалог). Блоки/линии с собственным `fontSize` (задан через контекстное меню) НЕ меняются – нельзя записывать число в сами узлы, иначе они станут кастомными.

Эффективный размер: `node.fontSize` → `sheet.defaultFontSize` → `SchemaLayoutMetrics.LABEL_FONT_SIZE`; для линии `edge.fontSize` → `sheet.defaultEdgeFontSize` → `EDGE_FONT`. Заведи один метод разрешения (`AppModel` или `schemalayout`) и используй его ВЕЗДЕ, где сейчас читается `getFontSize()`: `SchemaCanvasPanel.nodeTitleFont/nodeMetaFont` (~3830), раскладка гнёзд (~3887, ~3905), шрифт подписи линии (~2048), `AppModel` минимальный размер блока (~1607) и всё, что найдёшь `grep`'ом по `getFontSize`. После смены размера схемы – пересчитать минимальные размеры блоков схемы (как при смене шрифта блока), одна запись отмены. Пункт контекстного меню блока «Размер шрифта...» должен показывать, что 0 = «как у схемы».

Тесты: блок без `fontSize` берёт размер схемы, с `fontSize` – свой; то же для линии; смена размера схемы не пишет `fontSize` в узлы; минимальный размер блока растёт при увеличении шрифта схемы; `undo`.

Трек F — сохранение блока в библиотеку как пресета (пункт 9)

<Общие правила>

Проблема от пользователя: сейчас можно сохранить в библиотеку ПУСТОЙ пресет – кнопка « Сохранить как пресет» в форме «Добавить узел» справа от общей схемы (`SchemaPanel.saveAsPresetBtn / saveAsPreset`) берёт только подпись из поля ввода

и вызывает `addEquipmentPreset(mode, type, name, "", null)` без карт. Нужно: кнопку справа УБРАТЬ, сохранение перенести в меню ПКМ по блоку, и сохранять блок целиком – со всеми картами и разъёмами, которые пользователь добавил ему на схеме.

Файлы: `ui/SchemaPanel`, `ui/SchemaCanvasPanel` (`showNodeMenu`), `service/AppModel` (секция пресетов оборудования ~2400-2470), `ui/GuideDialog` (если там описана кнопка), тест.

- 1) `SchemaPanel`: удалить `saveAsPresetBtn`, `saveAsPreset()` и его включение в `updateAddFormEnablement`. Выбор существующего пресета в форме «Добавить узел» не трогать.
- 2) `AppModel.saveSchemaNodeAsPreset(SchemaNode node, String name, String description)` → `EquipmentPreset`: `mode` и `category` – от узла; карты (`node.getCards()`) и разъёмы питания (`node.getPowerConnectors()`) – глубокие копии со СВЕЖИМИ `id` (`duplicateCardWithFreshIds` / `duplicatePortWithFreshId` – те же, что при создании узла из пресета; порты внутри карт тоже с новыми `id`), все свойства портов (тип интерфейса/разъёма, направление, количество, фазы, автомат, роль и т.д.) сохраняются. Пустой узел (ни одной карты и ни одного разъёма) – `IllegalArgumentException` с русским текстом. Название не пустое. Если в ЛИЧНОЙ библиотеке уже есть пресет того же режима и категории с таким именем – отдельный метод `replaceEquipmentPresetContents(preset, node)` (заменяет карты/разъёмы, `id` пресета сохраняется); общие (`shared`) пресеты не заменять никогда. Раскладку гнезд (`PortPlacement`), размер, ориентацию и привязки узла в пресет НЕ переносить – у `EquipmentPreset` таких полей нет, `ledscheme-model` не меняем.
- 3) Меню ПКМ блока (`showNodeMenu`): пункт «Сохранить блок в библиотеку...». Показывать для обычных блоков оборудования; НЕ показывать для `SCREEN`, авто-легенд (`isAutoPortLegend/isAutoLineLegend`) и блоков сетевого оборудования (`networkDeviceTypeId != null` – у них своя библиотека). Для блока без карт и разъёмов пункт неактивен с подсказкой «Добавьте блоку карты или разъёмы – пустой пресет сохранить нельзя». По клику – диалог: название (по умолчанию подпись блока), описание (необязательно), строка «Будет сохранено: N карт, M портов, K разъёмов питания». При совпадении имени с личным пресетом – выбор «Заменить» / «Другое имя» / «Отмена». После сохранения – сообщение, где пресет искать (этап «Библиотеки», категория), и обновление списка пресетов в форме «Добавить узел».
- 4) `GuideDialog`: если кнопка описана – заменить на пункт меню ПКМ; если нет – добавить одно предложение в раздел про общую схему.

Тесты: сохранение узла с картами и разъёмами → пресет с теми же картами/портами/разъёмами (по содержимому), но другими `id`; обратный путь `addSchemaNodeFromPreset`(сохранённый пресет) даёт узел с тем же набором; пустой узел – исключение; замена существующего личного пресета сохраняет его `id`; общий пресет с тем же именем не заменяется; правка карт узла ПОСЛЕ сохранения не меняет пресет (глубокая копия).

Трек D — запуск с диска D: (пункт 7)

<Общие правила>

Проблема от пользователя: если папку программы (сборка `jpackage app-image, dist/AVE_ToolBox: AVE_ToolBox.exe + app/ + runtime/`) положить на диск D:, программа не открывается. В коде привязки к C: нет (см. `PLAN.md §3`). Детали симптома пользователь пришлёт отдельно – не жди, начни с того, что ниже.

- 1) Диагностика (на этой машине есть только C:). Эмулируй другой диск:
`subst X: <временная папка>` с копией `dist/AVE_ToolBox`, плюс варианты пути: с пробелами, с кириллицей (`X:\Программы\AVE ToolBox`), длинный путь. Запускай `exe`, проверяй, жив ли процесс через ~10 с, закрывай ТОЛЬКО по своему PID (никогда `taskkill /IM javaw` – у пользователя фоном работает `Minecraft`). Предупреди пользователя в отчёте, что открывались окна приложения. В конце убери `subst` и временные копии. Результаты – таблица «путь → запустилось/нет, ошибка».

- 2) Лог запуска (делать в любом случае): `App.main` – глобальный

Thread.setDefaultUncaughtExceptionHandler + запись стектрейса и сведений о среде (версия, java.home, user.dir, путь jar/лаунчера, jpackage.app-path, ОС) в ~/led-scheme/logs/startup.log (ротация: 5 файлов по 1 МБ, ошибки записи лога глушить). Существующий JOptionPane об ошибке запуска оставить и дописать в него путь к логу.

- 3) Перезапуск/автообновление (UpdateManager.restartCurrentJar, applyAndRestartWindows): если задан jpackage.app-path – перезапускать через него, а не через `javaw -jar` (в сборке jpackage системной Java может не быть); для bat-скрипта автообновления – то же. Поведение для запуска из обычного jar не менять.
- 4) Если диагностика нашла причину – исправь (флаги jpackage в SKILL release-client, код) и опиши. Если не воспроизводится – перечисли, что спросить у пользователя (точный путь, текст ошибки, съёмный/сетевой диск, содержимое startup.log) и как лог поможет.

Тесты: формирование команды перезапуска (с jpackage.app-path и без) – вынеси в чистую функцию; запись startup.log во временный home.

Трек E — веб-редактор масок (решение D4)

<Общие правила – кроме worktree: ledscheme-server НЕ git-репозиторий; перед правками сделай копию static/ в C:\Development\ledscheme-server\static-backup-2026-09-30. Деплой запрещён – только скилл deploy-server и только по отдельному явному разрешению пользователя.>

Предусловие: трек A влит (поведение масок в клиенте – эталон).

Файлы: C:\Development\ledscheme-server\src\main\resources\static\ (app.js, index.html, стили). Повторить в веб-редакторе масок (initMaskEditor и форма):

- 1) убрать растр (галочку и отрисовку);
- 2) галочки «Плашка» (имя) и «Разрешение» – как в клиенте, плашка рисуется при хотя бы одной;
- 3) «Экран-сетка» + «Высота x» (по умолчанию 2): маска выше в N раз, ячейки xN, подпись разрешения с «(сетка xN)»;
- 4) «Свои цвета...» – два input type=color, активны при выборе этого варианта. Сам массив MASK_PRESETS – ручная копия enum клиента (золотое правило 4), в него CUSTOM не добавлять.

Сверь визуально с клиентом (отрисовку одного и того же экрана). Проверка – открыть страницу локально в браузере (файл или простой статический сервер), скриншоты в отчёт.

Трек I — интеграция (волна 4)

<Общие правила – кроме: работаешь в основном дереве master; мерж веток claude/* в master разрешаю по одной, push – нет.>

- 1) Смержить по порядку A, D, F, C1, B, C2, C3, C4 (E – вне git). Конфликты разрешать по смыслу обеих сторон; после каждого мержа mvn -DskipTests package.
- 2) Полный mvn test – 0 failures.
- 3) CHANGELOG.md (раздел v2.6) и TESTS.md – из отчётов агентов; GuideDialog – тексты из отчётов A/C2/C3.
- 4) Скилл test-build: пересобирать testbuild/led-scheme-test.jar и запустить; пройти вручную (предупредить пользователя перед управлением рабочим столом): маски (растр нет, плашка/разрешение, сетка x2, свои цвета, предупреждение обрезки; канвас 20000x20000 – предупреждение ≥16k, экспорт, превью в пресетах 1/2...1/8 и Авто; ввод 30001 – отказ), схемы (две схемы питания, переименование, удаление, экспорт, спецификация с колонками, импорт в сетевой менеджер из второй схемы сигнала, шрифт схемы); спецификации – по файлу на сцену в папках сцен, колонки по схемам; ПКМ по блоку с картами → «Сохранить блок в библиотеку...» → новый узел из этого пресета с теми же картами; кнопки «Сохранить как пресет» справа больше нет.
- 5) Проверка совместимости D5: сохранить проект с тремя схемами, открыть JSON – схемы одного режима не пересекаются по X; открыть тем же клиентом – координаты на месте.

- 6) Обновить CLAUDE.md (строка docs/masks-and-schema-sheets в таблице документов уже есть – проверить актуальность), ARCHITECTURE.md (раздел про общую схему – несколько схем, сдвиг D5).
- 7) Отчёт пользователю: что готово, что проверено вживую, что осталось (деплой веб-страницы – отдельно по разрешению).

6. Риски и что проверить при приёмке

- **D5 и пересохранение старым клиентом:** разбивка на схемы теряется (узлы попадут в одну схему режима, разнесённые по X). Приемлемо по решению D5; упомянуть в CHANGELOG.
- **Спецификация без «Итого»** — осознанно (D1). Если пользователь захочет итог — отдельный запрос.
- **Общепроектной спецификации больше нет** (D6) — если кто-то собирал закупку по проекту целиком из одного файла, теперь это N файлов. Отметить в CHANGELOG.
- **Производительность потоковой записи:** канвас 30000×8000 — сотни МБ несжатых данных через Deflater; экспорт должен идти в фоне с прогрессом.
- **Экран шире 30000 px** (или выше с множителем «сетки») — его маска не экспортируется (D7), пользователь получает понятное сообщение; сам экран на «Сетапе» не ограничиваем.
- **Растр исчезнет и в уже экспортированных ранее форматах** (отдельные маски экранов) — это заказанное изменение, в CHANGELOG отметить.
- **Пункт 7 может не воспроизвестись** без данных пользователя — тогда главный результат трека D — лог запуска и исправленный перезапуск.